2026-5-12 C++vector--理论部分

- 1. initializer_list 的基本使用
- 2. vector 的基本操作与遍历
- 3. 复杂类型：struct A
- 4. push_back 的三种插入方式
- 5. emplace_back 的使用
- 6. 遍历 vector\ 的几种方式
- 7. 完整的性能对比示例
  - 总结

1. initializer_list 的基本使用

`initializer_list` 的底层是一个数组，有一个指向头的指针和一个指向尾的指针，解引用就是数据，加加就是下一个数据。

```
void test_initializer_list() {
    // 数组 vs initializer_list
    int a[] = {10, 20, 30};
    auto il1 = {10, 20, 30};
    auto il2 = {10, 20, 30, 2, 3, 4, 5, 6, 6};

    // 查看类型
    cout << typeid(il1).name() << endl;

    // 范围 for 遍历
    for (auto e : il1) {
        cout << e << " ";
    }
    cout << endl;

    for (auto e : il2) {
        cout << e << " ";
    }
    cout << endl;

    // 实际用途：初始化 vector
    vector<int> v1 = {10, 20, 30};  // 单参数的隐式类型转换，构造+拷贝构造，编译器优化成一个拷贝
    vector<int> v2({10, 20, 30});   // 上下两种写法都可
}
```

2. vector 的基本操作与遍历

```
void test_vector_basic() {
    vector<string> v;
    string s1("张张");

    v.push_back(s1);
    v.push_back("哈哈");

    // 范围 for 遍历，使用引用避免深拷贝
    for (auto& e : v) {
        cout << e << " ";
    }
    cout << endl;
}
```

3. 复杂类型：struct A

```
struct A {
    A(int a1, int a2) : _a1(a1), _a2(a2) {
        cout << "A(int a1,int a2)" << endl;
    }
    A(const A& aa) : _a1(aa._a1), _a2(aa._a2) {
        cout << "A(const A& aa)" << endl;
    }
    int _a1;
    int _a2;
};
```

4. push_back 的三种插入方式

```
void test_push_back_variants() {
    vector<A> v2;

    // 1. 构造传参
    A aa(1, 1);
    v2.push_back(aa);
```

```
    // 2. 传递匿名对象（会多一次拷贝构造）
    v2.push_back(A(1, 1));

    // 3. 隐式类型转换
    v2.push_back({1, 1});
}
```

---

5. emplace_back 的使用

`emplace_back` 功能上相当于 `push_back`，多用于复杂的结构体类型，可以直接传递构造参数，避免临时对象和拷贝。

```
void test_emplace_back() {
    vector<A> v2;

    // emplace_back 的各种用法
    v2.emplace_back(1, 1);       // ✅ 直接传递构造参数，效率最高
    v2.emplace_back(A(1, 1));    // 传递匿名对象
    v2.emplace_back(aa);          // 传递已有对象

    // ❌ 不支持这种写法
    // v2.emplace_back({1, 1});
}
```

---

6. 遍历 vector<A> 的几种方式

```
void test_traversal() {
    vector<A> v2;
    v2.emplace_back(3, 3);
    v2.emplace_back(2, 4);

    // 方式一：迭代器
    vector<A>::iterator it2 = v2.begin();
    while (it2 != v2.end()) {
        cout << it2->_a1 << ":" << it2->_a2 << endl;
        ++it2;
    }
    cout << endl;
```

```
    // 方式二：范围 for（推荐）
    for (auto& aa : v2) {
        cout << aa._a1 << ":" << aa._a2 << endl;
    }

    // 方式三：C++17 结构化绑定（注释示例）
    // for (auto [x, y] : v2) {
    //     cout << x << ":" << y << endl;
    // }
}
```

---

7. 完整的性能对比示例

```
void test_performance_compare() {
    vector<A> v1, v2;

    cout << "=== push_back 传临时对象 ===" << endl;
    v1.push_back(A(1, 2));   // 构造临时 + 拷贝构造

    cout << "=== emplace_back 直接构造 ===" << endl;
    v2.emplace_back(1, 2);   // 只调用构造函数
}
```

主函数调用示例

```
int main() {
    // test_initializer_list();
    // test_vector_basic();
    // test_push_back_variants();
    // test_emplace_back();
    // test_traversal();
    test_performance_compare();
    return 0;
}
```

总结

知识点	要点
`initializer_list`	底层是数组，支持 `{}` 初始化容器
`push_back`	传临时对象会多一次拷贝构造
`emplace_back`	直接传递构造参数，无临时对象，效率更高
遍历方式	迭代器、范围 for、结构化绑定（C++17）